BUĞRA KAAN ÇAKICI

MİCROPROCCESORS LABROTORY PROJECT RAPORT

Prototype of Test Bench For Propeller

I am part of a four-person team working on a TÜBİTAK 2209-A funded project focused on multicopter propeller design. My role involves managing the test systems and statistical analysis. Having collaborated with both university labs and industry partners, I noticed a deficiency in existing test setups. Although a full-scale system is not possible due to time and budget limits, I am planning to build a basic prototype. I will be personally financing the required components.

The aim of my project is measuring the thrust of propeller with using 3 Loadcell censor + 3X HX711 Amplifier + Ardiuno Uno + Breadboard + ESC+ Brushless motor + battery.

Module Breakdown

1. LoadCells: To measuring thrust value, I will use loadcell.
2. Amplifiers: To amplifies values coming from loadcells, I will use 3x HX
3. Motor: BLDC motor
4. Driver: Motor driver: 30 A Electronic Speed Controller ESC
5. İnterface: Serial monitor in PC for observing thrust data.

Hardware Choice

1-For C based microcontroller Ardiuno uno. ATMEGA

Workload Justification

The project involves three main workload components: mechanical design, electronic integration, and software development.

The mechanical structure will be designed and manufactured using a 3D printer. The load cells and motor will be mounted on a rigid profile to ensure stable and accurate force measurement.

On the electronic side, multiple load cells will be interfaced using individual HX711 amplifier modules, and their positions and connections will be carefully configured with the Arduino system.

From a software perspective, real-time data acquisition and processing will be implemented, and measurement data will be monitored via the serial interface.

The system is based on a multi-sensor measurement approach to improve accuracy and reliability. However, this introduces additional complexity, particularly in calibration, which requires individual sensor calibration, offset correction, and data fusion. I believe that I can produce basic prototype with using C based microcontroller.

Repository name on Github: bugrakaancakici-eee/220202021

Code for calibration factor.

For using loadcell I need to set calibration factor. Most load cells use Strain Gauges. When a load is applied, the body of the load cell deforms microscopically. This deformation changes the electrical resistance of the strain gauges mounted on it. Result of changing voltage signal is created. Creating signal amplifies with HX711 module than transmitted to microcontroller. Same time HX711 module is an Analog Digital Converter (ADC). İnput signal evaluate with calibrating factor in coding.

The calibration factor is a scaling coefficient that converts the raw numerical data from the HX711 into a real weight unit (grams, kg).

Producing Process

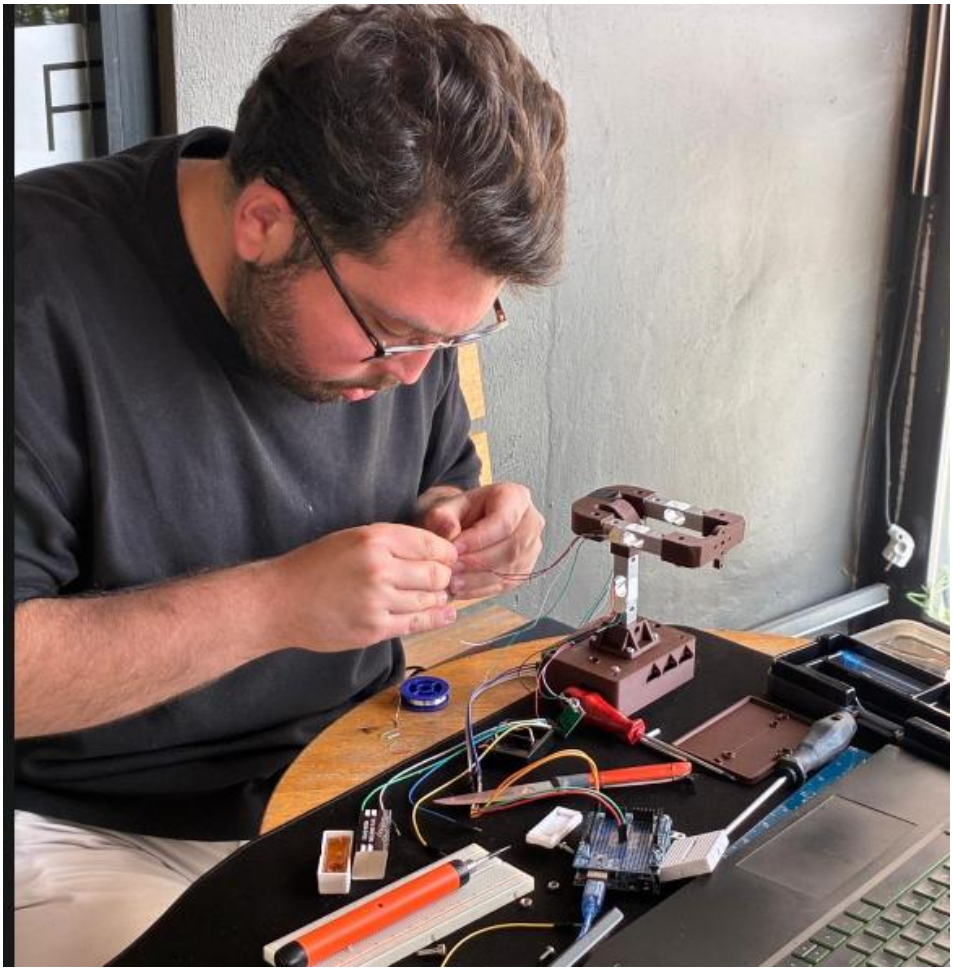

Figure 1 Soldering Step

The connections between the HX711 amplifiers and the load cells have been established via soldering. Calibration was performed prior to the final assembly. While the mechanical designs were sourced externally, the Arduino integration and firmware development were carried out by me.

```
#include "HX711.h"

// HX711 bağlantıları
#define DOUT 3
#define CLK 2

HX711 scale;

// Kalibrasyon faktörü
float calibration_factor = -418210;

// ================================================
// SETUP
// ================================================
void setup() {
  Serial.begin(9600);

  Serial.println("HX711 Kalibrasyonu");
  Serial.println("Tartidan tum agirliklari kaldirin.");
  Serial.println("Okumalar basladiktan sonra bilinen bir agirlik koyun.");
  Serial.println("Kalibrasyon faktorunu artirmak icin: a, s, d, f");
  Serial.println("Kalibrasyon faktorunu azaltmak icin: z, x, c, v");
  Serial.println("Tare / sifirlama icin: t");

  scale.begin(DOUT, CLK);

  scale.set_scale();
```

```
 scale.tare();

 long zero_factor = scale.read_average();

 Serial.print("Zero factor: ");
 Serial.println(zero_factor);
}

// ================================================
// LOOP
// ================================================
void loop() {
 scale.set_scale(calibration_factor);

 Serial.print("Okunan deger: ");
 Serial.print(scale.get_units(), 3);
 Serial.print(" kg");

 Serial.print(" | calibration_factor: ");
 Serial.println(calibration_factor);

 if (Serial.available()) {
  char temp = Serial.read();

  if (temp == '+' || temp == 'a') {
   calibration_factor += 10;
  }
  else if (temp == '-' || temp == 'z') {
   calibration_factor -= 10;
  }
  else if (temp == 's') {
```

```
    calibration_factor += 100;
   }
   else if (temp == 'x') {
    calibration_factor -= 100;
   }
   else if (temp == 'd') {
    calibration_factor += 1000;
   }
   else if (temp == 'c') {
    calibration_factor -= 1000;
   }
   else if (temp == 'f') {
    calibration_factor += 10000;
   }
   else if (temp == 'v') {
    calibration_factor -= 10000;
   }
   else if (temp == 't') {
    scale.tare();
    Serial.println("Tare yapildi. Terazi sifirlandi.");
   }
  }

  delay(500);
}
```

While setting calibration factor, references are iphone 11(167g), iphone 16(202g), 500ml watter bottle(500g) are used

```
17
18    Serial.println("HX711 Kalibrasyonu");
19    Serial.println("Tartidan tum agirliklari kaldirin.");
20    Serial.println("Okumalar basladiktan sonra bilinen bir agirlik koyun.");
21    Serial.println("Kalibrasyon faktorunu artirmak icin: a, s, d, f");
22    Serial.println("Kalibrasyon faktorunu azaltmak icin: z, x, c, v");
23    Serial.println("Tare / sifirlama icin: t");
24
25    scale.begin(DOUT, CLK);
26
27    scale.set_scale();
```

Output Serial Monitor ×

Message (Enter to send message to 'Arduino Uno' on 'COM9')

```
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.002 kg | calibration_factor: -418210.00
)kunan deger: -0.002 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibration_factor: -418210.00
)kunan deger: -0.001 kg | calibr
```

Figure 2 Calibrating, serial monitor

Thrust Bench Setting Without Motor Control

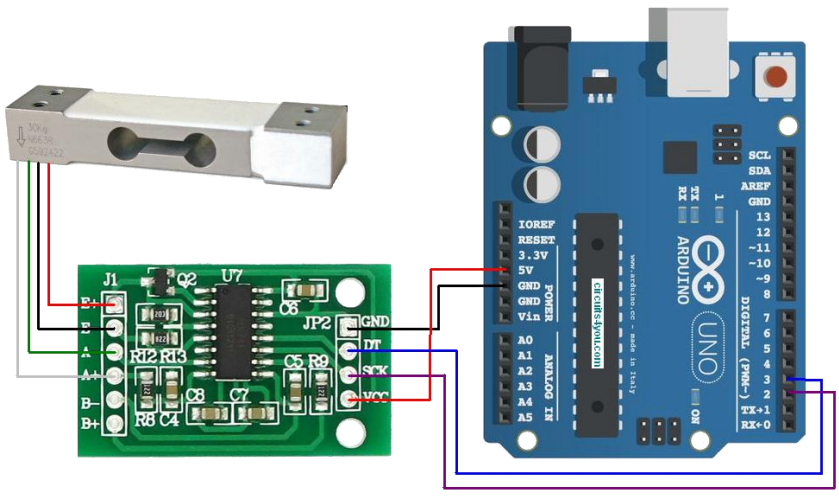

Figure 3 Arduino HX711 Connection

3 loadcell located to system

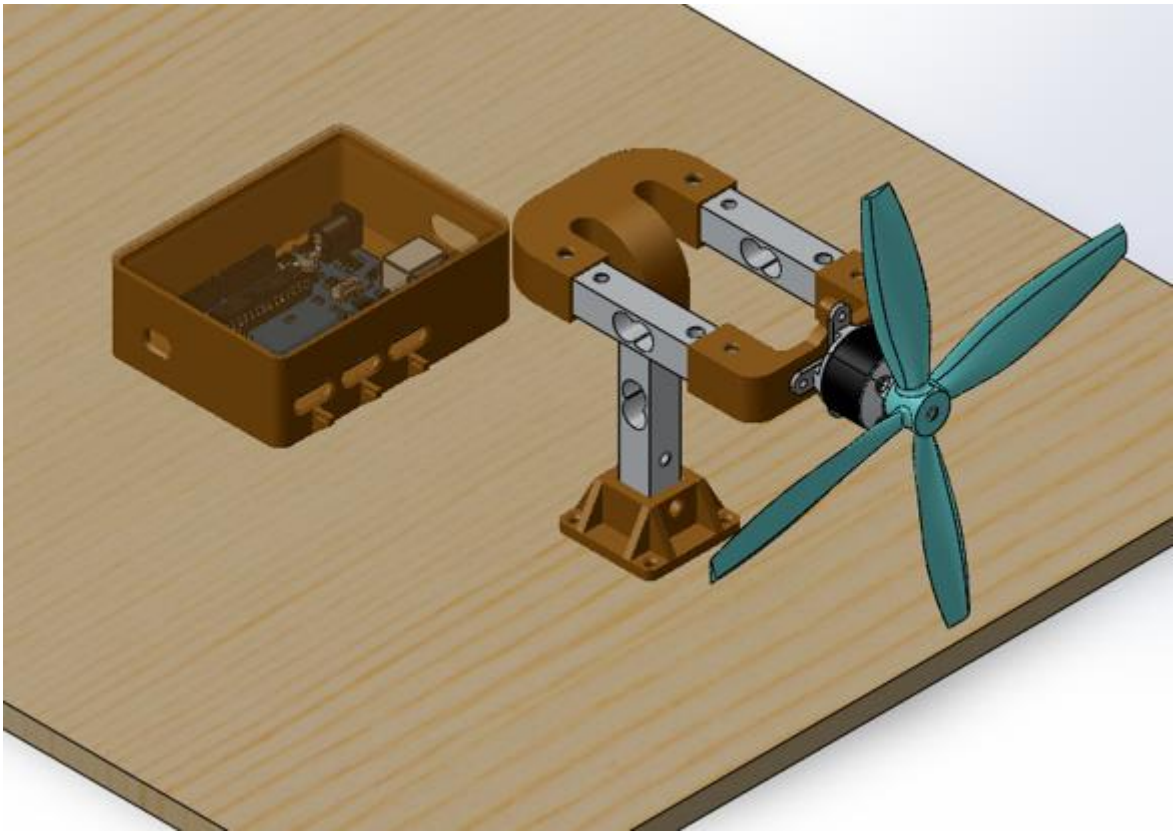

Figure 4 System SolidWorks

1st Loadcell for thrust measurement;

VCC→ 5V pin

GND → GND pin

DT → D3

SCK →D2

2nd loacell for torque measurement;

VCC→ 5V pin

GND → GND pin

DT → D7

SCK →D6

3th loadcell for torque measurement;

VCC→ 5V pin

GND → GND pin

DT → D5

SCK →D4

```
#include "HX711.h"

// 1. Load cell - Thrust
#define THRUST_DOUT 3
#define THRUST_CLK  2

// 2. Load cell - Torque side A
#define TORQUE1_DOUT 7
#define TORQUE1_CLK  6

// 3. Load cell - Torque side B
#define TORQUE2_DOUT 5
#define TORQUE2_CLK  4

HX711 thrustCell;
HX711 torqueCell1;
HX711 torqueCell2;

float thrust_calibration = -418210;
float torque1_calibration = -418210;
float torque2_calibration = -418210;
```

```
const float g = 9.81;
float torque_arm_m = 0.035;

const int sample_count = 10;

void setup() {
  Serial.begin(9600);

  thrustCell.begin(THRUST_DOUT, THRUST_CLK);
  torqueCell1.begin(TORQUE1_DOUT, TORQUE1_CLK);
  torqueCell2.begin(TORQUE2_DOUT, TORQUE2_CLK);

  thrustCell.set_scale(thrust_calibration);
  torqueCell1.set_scale(torque1_calibration);
  torqueCell2.set_scale(torque2_calibration);

  Serial.println("Propeller Thrust and Torque Test System");
  Serial.println("Remove all loads...");
  delay(1000);

  thrustCell.tare();
  torqueCell1.tare();
  torqueCell2.tare();

  Serial.println("Tare completed.");
  Serial.println("Press 't' to tare again.");
  Serial.println("-------------------------------------");
}

void loop() {
  float thrust_kg = thrustCell.get_units(sample_count);
```

```
float thrust_g = thrust_kg * 1000.0;
float thrust_N = thrust_kg * g;

float torque1_kg = torqueCell1.get_units(sample_count);
float torque2_kg = torqueCell2.get_units(sample_count);

float torque1_N = torque1_kg * g;
float torque2_N = torque2_kg * g;

float torque_Nm = (torque1_N - torque2_N) * torque_arm_m;

Serial.println("---------------------------------------");

Serial.print("Time: ");
Serial.print(millis());
Serial.println(" ms");

Serial.print("Thrust: ");
Serial.print(thrust_N, 4);
Serial.print(" N  |  ");
Serial.print(thrust_g, 2);
Serial.println(" g");

Serial.print("Torque Cell 1: ");
Serial.print(torque1_N, 4);
Serial.println(" N");

Serial.print("Torque Cell 2: ");
Serial.print(torque2_N, 4);
Serial.println(" N");
```

```
  Serial.print("Torque: ");
  Serial.print(torque_Nm, 5);
  Serial.println(" N.m");

  if (Serial.available()) {
    char command = Serial.read();

    if (command == 't') {
      thrustCell.tare();
      torqueCell1.tare();
      torqueCell2.tare();
      Serial.println("All load cells tared.");
    }
  }

  delay(50);
}
```

FULL CODE: WİTH MOTOR CONTROL

```
#include "HX711.h"
#include <Servo.h>

// ================================
// HX711 PINLERİ
// ================================

// 1. Load cell - Thrust
#define THRUST_DOUT 3
#define THRUST_CLK  2
```

```
// 2. Load cell - Torque side A
#define TORQUE1_DOUT 7
#define TORQUE1_CLK  6

// 3. Load cell - Torque side B
#define TORQUE2_DOUT 5
#define TORQUE2_CLK  4

// ESC signal pin
#define ESC_PIN 9

HX711 thrustCell;
HX711 torqueCell1;
HX711 torqueCell2;

Servo esc;

// ================================
// KALİBRASYON FAKTÖRLERİ
// ================================

float thrust_calibration = -418210;
float torque1_calibration = -418210;
float torque2_calibration = -418210;

// ================================
// SABİTLER
// ================================

const float g = 9.81;
```

```
float torque_arm_m = 0.035;

const int sample_count = 10;

// ESC pulse değerleri
const int ESC_MIN = 1000;
const int ESC_MAX = 2000;

int throttle_percent = 0;
int esc_signal = ESC_MIN;

void setup() {
  Serial.begin(9600);

  // Load cell başlatma
  thrustCell.begin(THRUST_DOUT, THRUST_CLK);
  torqueCell1.begin(TORQUE1_DOUT, TORQUE1_CLK);
  torqueCell2.begin(TORQUE2_DOUT, TORQUE2_CLK);

  thrustCell.set_scale(thrust_calibration);
  torqueCell1.set_scale(torque1_calibration);
  torqueCell2.set_scale(torque2_calibration);

  // ESC başlatma
  esc.attach(ESC_PIN);
  esc.writeMicroseconds(ESC_MIN);

  Serial.println("Propeller Thrust, Torque and ESC Control System");
  Serial.println("Remove propeller for first test!");
  Serial.println("Arming ESC...");
  delay(3000);
```

```
  Serial.println("Remove all loads...");
  delay(1000);

  thrustCell.tare();
  torqueCell1.tare();
  torqueCell2.tare();

  Serial.println("Tare completed.");
  Serial.println("-------------------------------------");
  Serial.println("Commands:");
  Serial.println("0 = STOP");
  Serial.println("1 = 10% throttle");
  Serial.println("2 = 25% throttle");
  Serial.println("3 = 50% throttle");
  Serial.println("4 = 75% throttle");
  Serial.println("5 = 100% throttle");
  Serial.println("t = tare load cells");
  Serial.println("-------------------------------------");
}

void loop() {
  // ================================
  // SERIAL KOMUT OKUMA
  // ================================

  if (Serial.available()) {
    char command = Serial.read();

    if (command == '0') {
      throttle_percent = 0;
```

```
  }
  else if (command == '1') {
    throttle_percent = 10;
  }
  else if (command == '2') {
    throttle_percent = 25;
  }
  else if (command == '3') {
    throttle_percent = 50;
  }
  else if (command == '4') {
    throttle_percent = 75;
  }
  else if (command == '5') {
    throttle_percent = 100;
  }
  else if (command == 't') {
    thrustCell.tare();
    torqueCell1.tare();
    torqueCell2.tare();
    Serial.println("All load cells tared.");
  }

  esc_signal = map(throttle_percent, 0, 100, ESC_MIN, ESC_MAX);
  esc.writeMicroseconds(esc_signal);
}

// ================================
// THRUST ÖLÇÜMÜ
// ================================
```

```
float thrust_kg = thrustCell.get_units(sample_count);
float thrust_g = thrust_kg * 1000.0;
float thrust_N = thrust_kg * g;

// ================================
// TORQUE ÖLÇÜMÜ
// ================================

float torque1_kg = torqueCell1.get_units(sample_count);
float torque2_kg = torqueCell2.get_units(sample_count);

float torque1_N = torque1_kg * g;
float torque2_N = torque2_kg * g;

float torque_Nm = (torque1_N - torque2_N) * torque_arm_m;

// ================================
// SERIAL MONITOR ÇIKTISI
// ================================

Serial.println("-----------------------------------");

Serial.print("Throttle Mode: ");
Serial.print(throttle_percent);
Serial.print("%  | ESC Signal: ");
Serial.print(esc_signal);
Serial.println(" us");

Serial.print("Thrust: ");
Serial.print(thrust_N, 4);
Serial.print(" N  |  ");
```

```
  Serial.print(thrust_g, 2);
  Serial.println(" g");

  Serial.print("Torque Cell 1: ");
  Serial.print(torque1_N, 4);
  Serial.println(" N");

  Serial.print("Torque Cell 2: ");
  Serial.print(torque2_N, 4);
  Serial.println(" N");

  Serial.print("Torque: ");
  Serial.print(torque_Nm, 5);
  Serial.println(" N.m");

  delay(300);
}
```